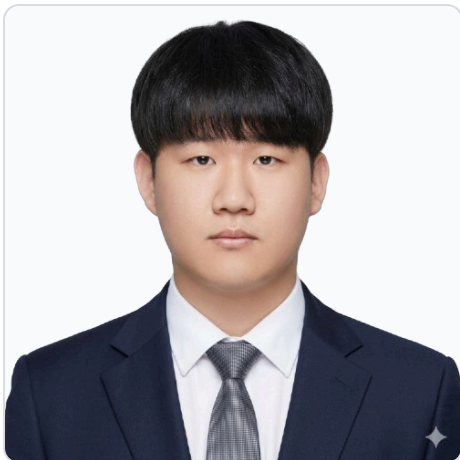


PORTFOLIO PROFILE



손지유

sonjiyu

**AI와 함께 주어진 환경을 빠르게 파악하고,
필요한 역할을 수행해 결과로 연결하는 개발
자**

GITHUB

github.com/guson401

PORTFOLIO WEB

guson401.github.io

EMAIL

sonjiyu123@gmail.com

PHONE

010-8245-3525

CONTENTS

Project Portfolio

웹, 모바일, AIoT 프로젝트를 하나의 포트폴리오로 정리했습니다. 표지 다음 페이지부터는 각 프로젝트 상세 페이지를 사용해 주요 역할, 구현 내용, 트러블 슈팅, 아키텍처를 확인할 수 있습니다.

01

Omni-Kit

STT 기반 협동 로봇팔 제어와 터치펜던트 대체 웹 콘솔 프로젝트

React

Spring Boot

WebAuthn

02

DailyLog

대화형 일기 작성, 감정 분석, 캘린더 연동을 제공하는 Android 앱 프로젝트

Kotlin

Compose

Android

03

Meer's Farm

병해충 감시 자율주행 로봇과 농장 관제 웹 서비스를 연결한 AIoT 프로젝트

React

Django

MQTT

PROJECT OVERVIEW

Omni-Kit

OnRobot Korea와 함께한 연계 프로젝트로, STT를 이용한 협동 로봇팔 제어와 기존 티치펜던트를 대체할 웹 콘솔 구현을 목표로 했습니다. 웹 내부는 REST/WebSocket으로 구성하고, 로봇과 웹 시스템은 MQTT 토픽으로만 소통합니다.

PROJECT SNAPSHOT

React 19

Vite 6 + TS · URDF 디지털 트윈

Spring 3

JDK 21 · JPA · Redis 설계

Device Guard

JWT · WebAuthn · 화이트리스트

Robot Link

MQTT · UR5/2FG7 연동

문제 정의

기존 협동 로봇팔 제어는 티치펜던트와 현장 PC 중심이라 음성 명령이나 웹 기반 운영 흐름으로 확장하기 어렵습니다. Omni-Kit은 STT 명령, 로봇 상태, 작업 흐름을 웹 콘솔로 끌어올려 더 직관적인 운영 환경을 만드는 프로젝트입니다.

핵심 구현 포인트

React 콘솔은 STT 음성 패널, URDF 3D 모델, 레시피, 웨이포인트, 세션·장치 관리 화면을 제공하고, Spring Boot는 인증·운영 API와 데이터 모델을 담당합니다. 로봇과 웹 시스템 사이의 송수신은 MQTT 토픽으로 처리했습니다.

01

웹 콘솔

상태 확인, STT 패널, TCP/조인트 제어, 레시피·웨이포인트·작업 이력을 React 관리자 콘솔로 묶었습니다.

02

웹 파트 설계

프론트엔드 화면·라우팅·인증 흐름과 Spring Boot API, 도메인, 배포/인프라 구조를 정리했습니다.

03

WebAuthn

Yubico WebAuthn 기반 장치 등록·인증·승인 모델로 승인된 하드웨어만 접근하도록 보호했습니다.

04

MQTT 연동

로봇 파트의 상태·이벤트 형식에 맞춰 토픽 매핑과 화면 반영을 최소 수정으로 맞췄습니다.



Omni-Kit

로봇 STT 기반 업무로봇 제어 플랫폼

로그인

사이트 코드

SITE-SIM

사원번호

A-1001

비밀번호

로그인

비밀번호 찾기

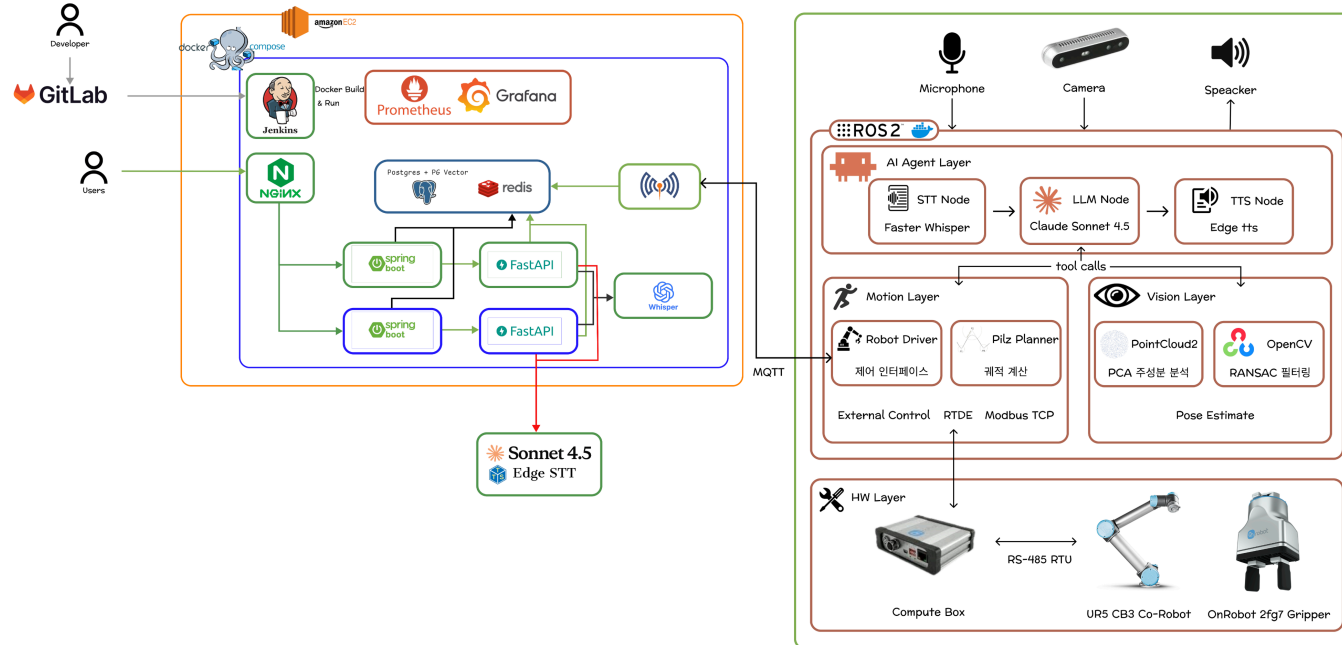
PC : 태블릿과 admin / manager 전용. 작업자(operator)는 모바일 앱에서만 접속 가능합니다.

사이트 코드와 사원으로 로그인한 뒤 저장된 패스키를 선택하고, WebAuthn 디바이스 승인 상태를 확인하는 진입 흐름입니다.

패스키 기반 인증 후 콘솔 진입

등록 장치 정보와 현재 브라우저 fingerprint 대조

세션 갱신 실패 같은 예외 복구 동선 제공



웹 내부는 Nginx, React, Spring Boot, FastAPI, PostgreSQL/Redis로 구성되며, 로봇 시스템과는 Mosquitto MQTT 토픽으로만 상태와 명령을 주고받습니다.

웹 · 백엔드 · 인프라 영역

사용자는 Nginx를 통해 React 콘솔에 접근하고, 콘솔은 Spring Boot API/WS와 통신합니다. 백엔드는 인증, 세션, 운영 도메인, DB 모델을 담당합니다.

경계 분리

로봇과 웹 시스템은 REST나 WebSocket으로 직접 연결되지 않고 MQTT 토픽으로만 소통합니다.

역할

프론트엔드 전체 구현

로그인, 승인 대기, 대시보드, 실시간 제어, 로봇/레시퍼/웨이포 인트/이력, 관리자 화면과 라우팅/인증 가드를 구현했습니다.

백엔드 · DB 설계

회원, 사이트, 장치, 세션, 알람, MFA 도메인의 API와 JPA 모델, Redis 기반 challenge 흐름을 정리했습니다.

WebAuthn 화이트리스트

Yubico RelyingParty 기반 등록 옵션·검증·어설션과 승인 credential 기반 접근 제어를 구성했습니다.

인프라 · MQTT 연동

웹/백엔드 배포 구조와 환경 변수를 정리하고, 로봇과 웹 사이 통신 범위를 MQTT 중심으로 맞췄습니다.

트러블슈팅

01 takeover 기반 단일 세션 정책 보정

기존 탭이 계속 살아 있던 문제를 takeover 로그인 시 기존 세션 무효화와 /auth/me 재검증으로 해결했습니다.

02 Linux WebAuthn 개발 우회

Linux platform authenticator 부재를 고려해 실서비스 접근 제한과 개발용 User-Agent 우회 경로를 분리했습니다.

03 소프트 삭제 기기 재승인 흐름

폐기된 기기의 재등록이 막히던 문제를 승인 대기 상태로 되돌려 재검토할 수 있게 수정했습니다.

사용기술

Frontend / Console UI

- React 19.2
- Vite 6.4
- TypeScript
- React Router
- TanStack Query
- three.js

Backend / Auth / DB

- Spring Boot 3.5
- JDK 21
- Yubico WebAuthn
- JWT
- JPA
- Redis

Robot Communication / Infra

- MQTT
- FastAPI
- Docker Compose
- Nginx
- Prometheus
- Grafana

PROJECT OVERVIEW

DailyLog

STT/TTS 기반 일기 작성 어플리케이션 프로젝트입니다. 대화 기반으로 하루를 기록하고 감정 결과를 일정 추천과 캘린더 흐름으로 연결하는 Android 앱입니다.

PROJECT SNAPSHOT

Android

Jetpack Compose UI

Diary

감정 기록과 결과 화면 구성

Calendar

Calendar Provider 유지

Flow

실사용 흐름 기반 화면 구성

문제 정의

하루의 정리를 대화 기반으로 더 간단하게 기록하고, 그 결과를 바탕으로 일정 추천과 일정 관리까지 자연스럽게 이어질 수 있는 경험을 만들고자 했습니다.

일기 작성

질문형 흐름을 따라 하루의 감정과 상황을 기록하고, 결과 화면까지 자연스럽게 이어지도록 구성했습니다.

감정 기반 일정 추천

분석된 감정 결과를 바탕으로 추천 활동을 제안하고 일정과 연결해 다음 행동으로 이어지게 했습니다.

핵심 구현 포인트

Compose 화면 설계, Repository 계층, Room DB, 캘린더 프로바이더 연동, 결과 페이지와 프로필 그래프 흐름 검증을 중심으로 작업했습니다.

일기 기반 감정 분석

작성한 일기 내용을 바탕으로 감정 결과를 분석하고 사용자가 하루를 돌아볼 수 있도록 피드백을 제공합니다.

캘린더 연동

앱 내부 기록과 Calendar Provider 이벤트를 함께 보여주어 감정 기록과 일정 관리를 한 흐름으로 연결합니다.



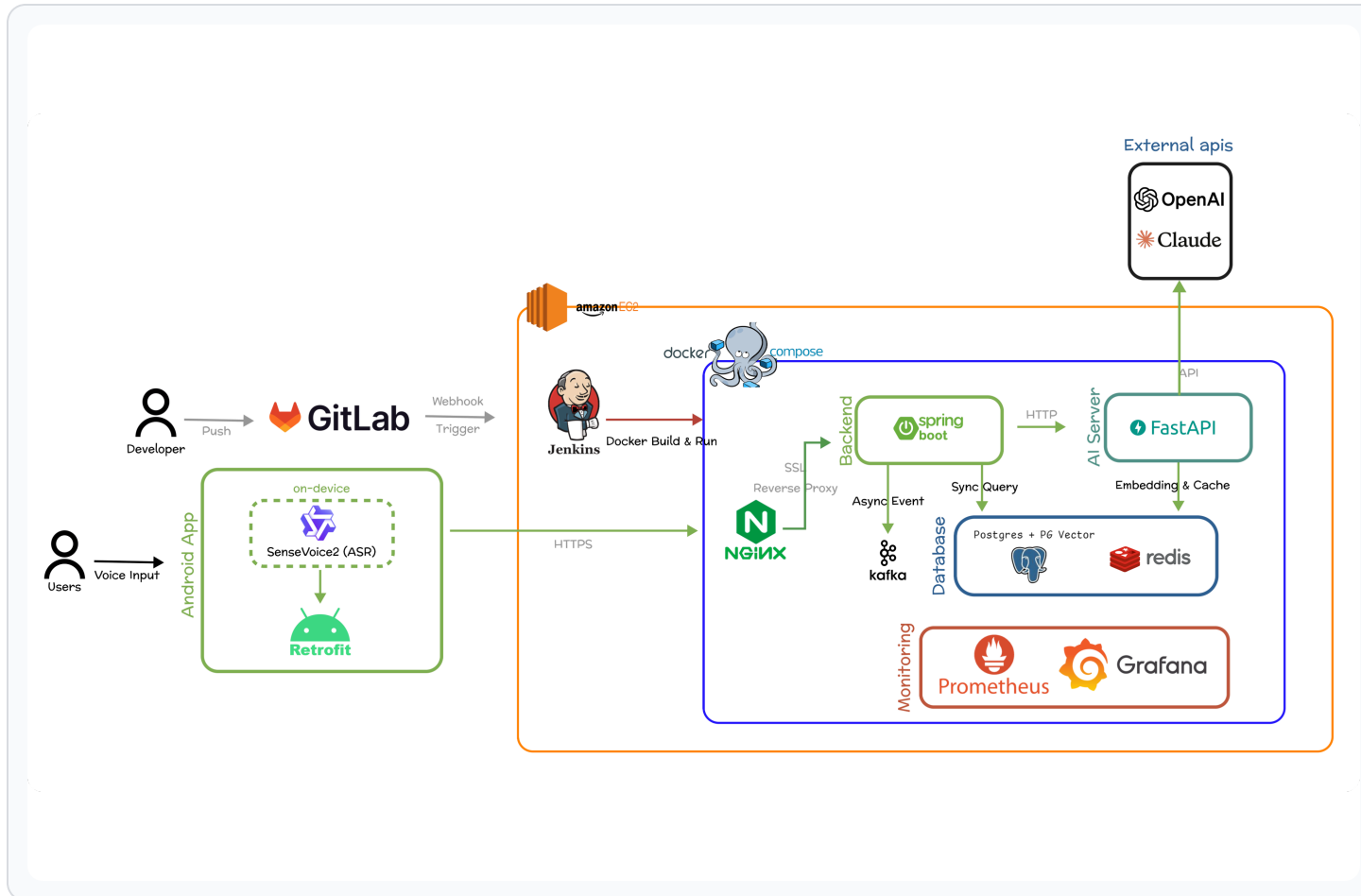
AUTH

앱 사용을 시작할 때 가장 먼저 진입하는 인증 화면으로, 로그인 실행뿐 아니라 비밀번호 찾기와 회원가입 흐름으로도 분기되는 출발점입니다.

인증 화면까지 포함해 전체 사용자 여정 완성

회원 흐름의 시작점과 복귀 지점 설명

앱 전반의 완결성을 보강하는 화면



DailyLog는 Android 앱에서 음성 입력과 화면 흐름을 처리하고, 서버는 API 처리, 데이터 저장, AI 연동, 배포와 모니터링을 분리해 운영할 수 있도록 구성했습니다.

Android App

사용자 음성 입력은 온디바이스 ASR로 처리하고, 앱 내부 화면은 Retrofit을 통해 HTTPS API와 연결했습니다.

Backend & AI

Spring Boot, PostgreSQL/PGVector, Redis, FastAPI AI 서버와 배포/모니터링 흐름을 분리했습니다.

역할

화면 흐름 설계

로그인, 회원가입, 튜토리얼, 홈, 일기 작성, 결과, 캘린더, 프로필 까지 이어지는 주요 흐름을 설계했습니다.

로컬 데이터 구조

Room DB와 Repository 구조를 통해 일기와 사용자 데이터를 안정적으로 저장하고 계층을 분리했습니다.

Compose 기반 UI

오브젝트 중심 인터페이스 감성을 유지하면서 각 화면의 레이아웃과 상호작용을 구현했습니다.

캘린더/프로필 검증

일기 작성, 감정 분석, 추천, Calendar Provider 연동, 감정 그래프까지 실제 흐름으로 검증했습니다.

트러블슈팅

01 클릭 뎁스를 줄이기 위한 UI 수정

자주 쓰는 일기 작성, 일기 확인, 캘린더 진입을 홈 메뉴에서 바로 선택하도록 재배치했습니다.

02 상호작용형 UI 튜토리얼

오브젝트 기반 화면에서 놀려야 하는 요소와 다음 흐름을 단계별로 보여주어 초기 학습 비용을 낮췄습니다.

사용기술

UI / Android

- Kotlin
- Jetpack Compose
- Material 3

Data / Storage

- Room DB
- Repository Pattern
- Kotlin Serialization

Network / Device

- Retrofit
- OkHttp
- Calendar Provider
- ASR
- TTS

PROJECT OVERVIEW

Meer's Farm

센서 데이터, 로봇 순찰, 병해충 탐지, 운영 리포트를 하나의 흐름으로 연결한 스마트팜 통합 관제 웹 서비스입니다. 조회에서 끝나지 않고 상태 확인 이후 원인 파악과 제어 액션까지 이어지도록 구성했습니다.

PROJECT SNAPSHOT

React

SPA UI Flow

Django

REST API

MQTT

Sensor Update

Report

Image + Log Linkage

문제 정의

농부가 병해충 여부와 식물 상태를 직접 확인해야 하는 부담을 줄이고, 로봇 순찰과 분석 결과를 짧은 시간 안에 이해할 수 있게 하는 데 초점을 맞췄습니다.

통합 대시보드

전체 농장 보기와 개별 농장 보기를 나누어 온도, 습도, 조도, pH, NPK, 위험도 데이터를 빠르게 비교할 수 있게 했습니다.

로봇 제어

로봇 등록, 상세 조회, 모드 전환, 정지, 복귀, 스케줄 타임라인까지 연결해 관제 기능을 강화했습니다.

핵심 구현 포인트

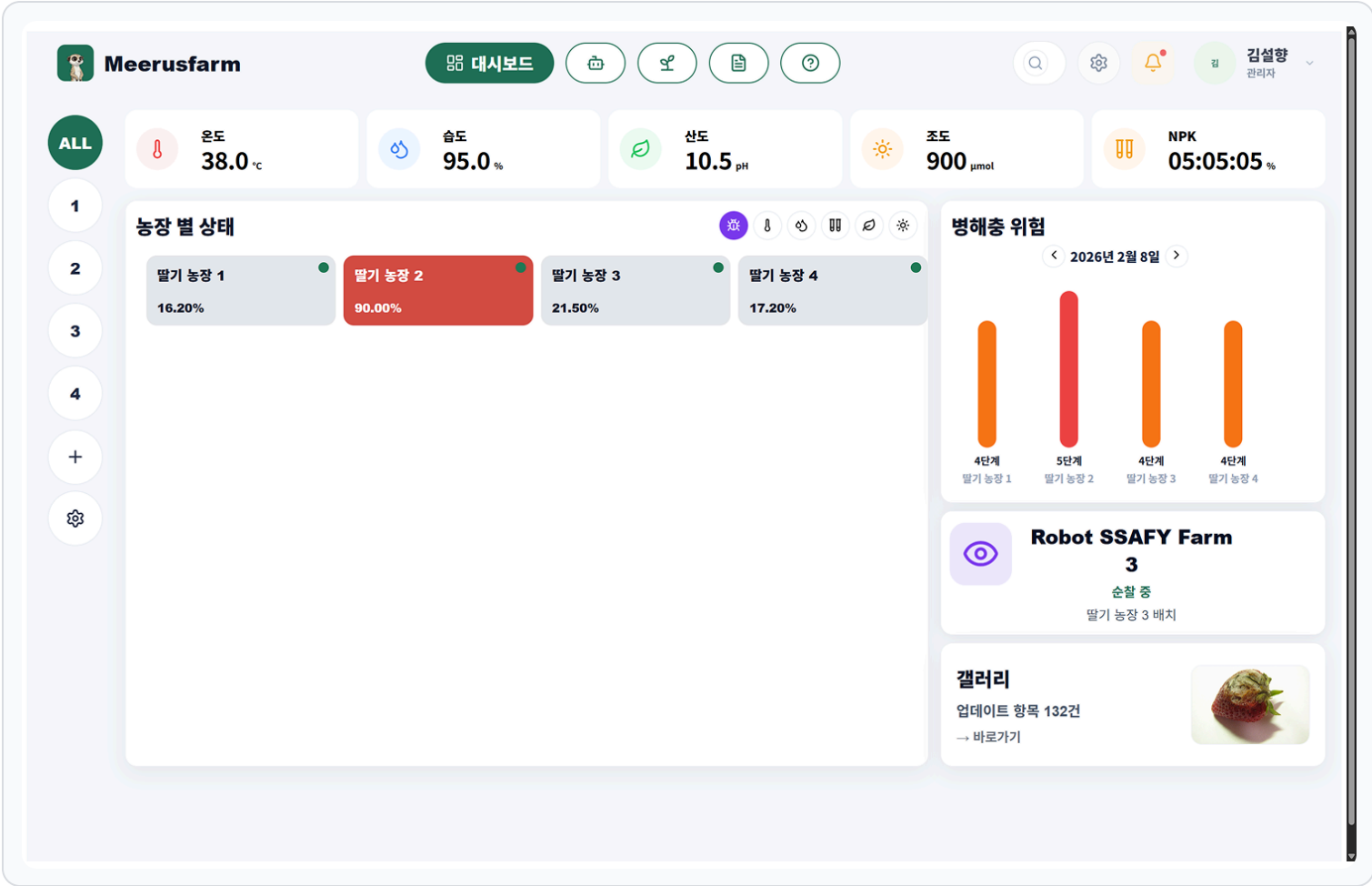
대시보드, 농장 상세, 로봇 제어, 이미지 갤러리, 운영 리포트를 하나의 서비스 흐름으로 연결했습니다.

농장/섹터 관리

행, 열, 층수 기반 구조 데이터와 자동화 임계값을 함께 다루고 섹터 상태가 상위 대시보드에 반영되도록 설계했습니다.

이미지/리포트 연계

병해충 이미지와 리포트를 시간 기준으로 매칭해 어떤 문제가 언제 발생했는지 함께 확인할 수 있게 구성했습니다.

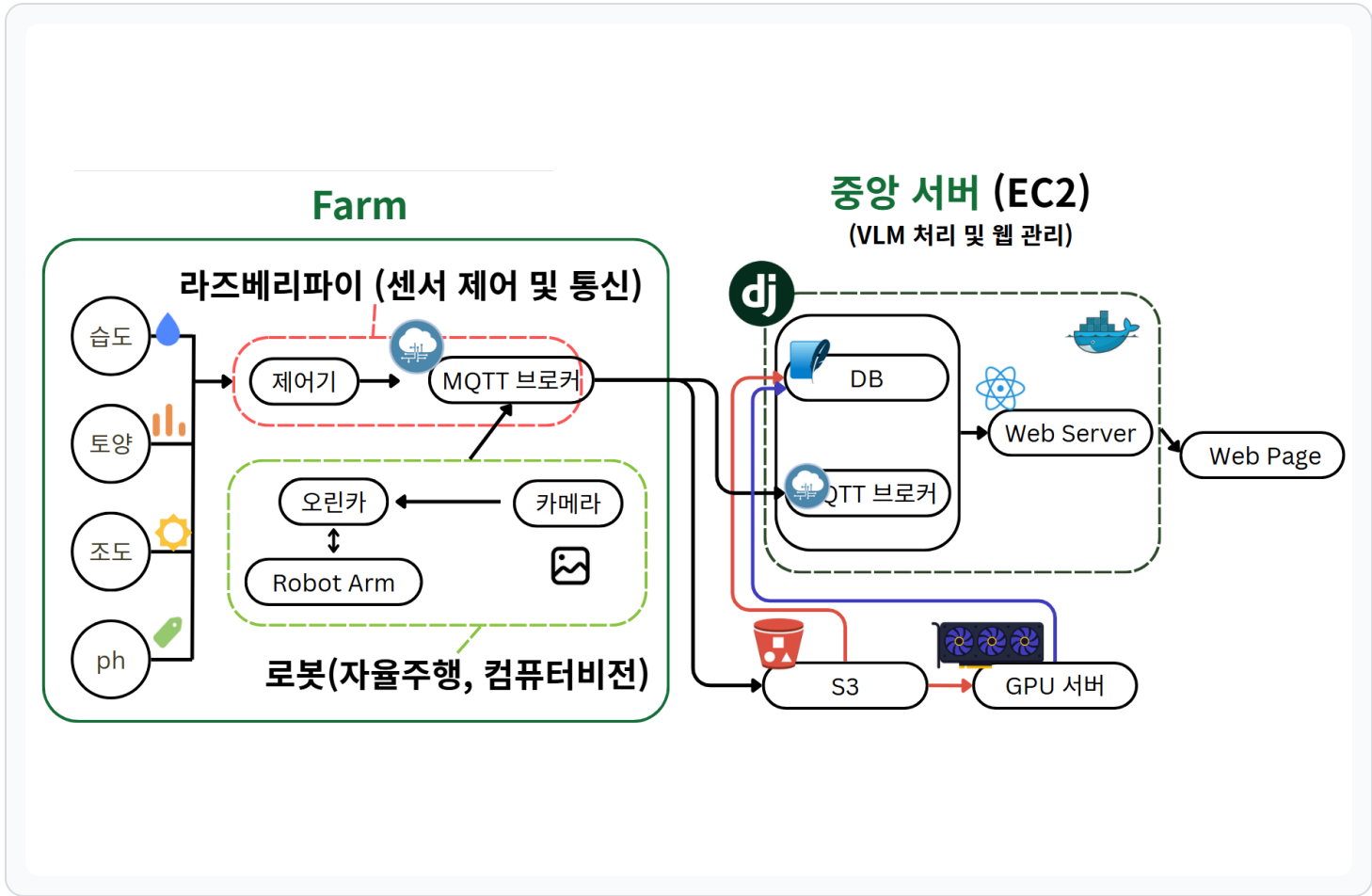


전체 농장 기준으로 셀 상태가 활성화된 대시보드 화면입니다.

전체 농장 단위 모니터링 화면

상위 레벨에서 이상 징후를 먼저 파악

운영 판단의 출발점이 되는 화면



Farm 현장의 라즈베리파이와 로봇에서 발생한 센서/이미지 데이터를 MQTT와 저장소 흐름으로 중앙 서버에 모으고, 사용자는 웹 페이지에서 상태와 리포트를 확인하는 구조입니다.

Farm Edge · Robot

라즈베리파이, 센서, 카메라, 로봇암, S3, GPU 서버의 VLM 처리 흐름을 웹 관제로 연결했습니다.

Central Web

EC2 중앙 서버에서 Django, DB, MQTT 브로커, React 웹 서버를 운영해 센서 상태와 분석 결과를 반영합니다.

역할

API 설계 및 구현

Django REST Framework 기반 농장, 섹터, 로봇, 리포트 API를 설계했습니다.

대시보드/리포트 UI

React 기반 대시보드, 리포트, 갤러리, 로봇 상세 화면을 구현했습니다.

인증 및 권한 처리

JWT 로그인, 회원가입, 사용자 설정과 접근 제어 로직을 적용했습니다.

데이터 연결 구조

병해충 이미지, 리포트, 로그를 시간 기준으로 연결했습니다.

트러블슈팅

01 배포 환경 파일명 오류 해결

Windows 로컬과 Linux 배포 환경의 파일명 대소문자 차이를 확인하고 CSS 파일명과 import 경로를 소문자 기준으로 통일했습니다.

02 더미 데이터에서 실제 API로 전환

farmId/id, createdAt/create_time, NPK 객체 구조 차이를 화면용 공통 포맷으로 정규화했습니다.

03 클릭 뎀스를 줄인 UI 재설계

대시보드와 상단 메뉴에 농장/로봇 상세 이동, 갤러리 바로가기, 농장 추가 진입점을 배치했습니다.

사용기술

Frontend / Web UI

- React
- Vite
- React Router
- Axios
- Tailwind CSS

Backend / API / Auth

- Django
- Django REST Framework
- Simple JWT
- SQLite

Data / Robot / Infra

- MQTT
- S3
- GPU Server / VLM
- EC2